



Generalized synchronization between two distinct chaotic systems through deep reinforcement learning

Haoxin Cheng^a, Haihong Li^{a,b,*}, Jinfeng Liang^a, Qionglin Dai^{a,b}, Junzhong Yang^{a,*}

^a School of Physical Science and Technology, Beijing University of Posts and Telecommunications, Beijing, 100876, People's Republic of China

^b Key Laboratory of Mathematics and Information Networks (Beijing University of Posts and Telecommunications), Ministry of Education, People's Republic of China

ARTICLE INFO

Keywords:

Generalized chaotic synchronization
Model-free method
Deep reinforcement learning
Continuous control

ABSTRACT

This study presents a novel model-free deep reinforcement learning (DRL) approach to control generalized chaotic synchronization between two distinct chaotic systems, referred to as the slave and master systems. Our method enables an agent to progressively optimize its strategy for applying external compensation to the slave system through continuous interaction with both systems, achieving generalized synchronization without requiring prior knowledge of the underlying chaotic dynamics. We validate our approach on several classical chaotic systems, including the Lorenz system, the Rössler system, and the Chua circuit, demonstrating rapid synchronization and exceptional performance. A key contribution of this work is the ability of a single learned agent to control generalized chaotic synchronization across various slave–master system configurations. The DRL approach shows remarkable robustness, maintaining effective synchronization control even with limited information about the state variables of both systems. Notably, our method successfully controls high-dimensional coupled chaotic systems while requiring only minimal state variable information. The results underscore the potential of DRL for applications in secure communication and other fields where chaos control is crucial.

1. Introduction

Chaotic systems are known for their extreme sensitivity to initial conditions, where even minute variations in starting points can lead to drastically different outcomes over time. This characteristic makes achieving chaos synchronization—a phenomenon in which the dynamic behaviors of two or more chaotic systems align under certain conditions—a highly challenging task. In 1990, Pecora and Carroll [1] demonstrated chaos synchronization using a drive-response scheme in the Rössler and Lorenz systems. This groundbreaking work sparked significant advancements in the field [2–6], with chaos synchronization being applied in various domains, particularly in secure communications [7–9]. Notably, it has been used for image encryption [10–13] and voice encryption [14], enhancing the security of transmitted information.

The spectrum of chaos synchronization includes several types [15], such as complete synchronization [16–19], phase synchronization [20–22], lag synchronization [5], and generalized synchronization [23]. This study focuses on generalized synchronization, a concept introduced by Rulkov et al. [24], which describes a state in which a functional relationship is established between the state variables of

the driving and response systems. Since its introduction, significant progress has been made in understanding and controlling generalized synchronization. Notably, Hramov et al. [25] identified regions of generalized chaotic synchronization in unidirectionally coupled one-dimensional Ginzburg–Landau equations, shedding light on the mechanisms behind generalized synchronization in spatially extended chaotic systems. Deng [26] provided an in-depth analysis of the mechanisms of generalized synchronization in fractional-order systems, establishing necessary and sufficient conditions and discussing methods for detection. Zhang et al. [27] presented sufficient conditions for determining generalized synchronization in continuous systems using Lyapunov stability theory and designing appropriate unidirectional coupling terms. Huang et al. [28] developed sufficient conditions for generalized lag synchronization in continuous chaotic systems.

These studies assume prior knowledge of the systems' dynamical equations. However, situations where such information is unavailable are quite common. To address this challenge, we propose a model-free control approach that utilizes deep reinforcement learning (DRL) to achieve generalized chaotic synchronization between two distinct chaotic systems, even in the absence of such information. Generalized

* Corresponding authors.

E-mail addresses: haihongli@bupt.edu.cn (H.H. Li), jzyang@bupt.edu.cn (J.Z. Yang).

<https://doi.org/10.1016/j.chaos.2025.116727>

Received 28 November 2024; Received in revised form 7 March 2025; Accepted 8 June 2025

Available online 30 June 2025

0960-0779/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

chaotic synchronization can be classified into strong synchronization and weak synchronization [29]. Strong synchronization refers to a high real-time correlation between the state variables of the two systems, following a mapping relationship and resulting in minimal synchronization error. In contrast, weak synchronization suggests that the state variables may not be identical but exhibit certain similarities. We demonstrate that a single trained agent can achieve strong generalized chaotic synchronization across a range of different chaotic systems, and that this approach remains robust even with incomplete state information.

Reinforcement learning (RL) is a key branch of machine learning, allowing agents to learn optimal decision-making strategies through trial-and-error interactions with their environment. These interactions are guided by a reward signal that evaluates actions and steers the agent toward favorable outcomes. RL is particularly effective at solving sequential decision-making problems in dynamic and uncertain environments [30,31]. Deep reinforcement learning (DRL), which combines RL with deep learning, has expanded the scope of RL by enabling agents to manage high-dimensional state and action spaces. DRL algorithms leverage deep neural networks to approximate value functions, policies, or environment dynamics, allowing them to solve problems that were previously computationally infeasible. This combination has led to remarkable successes across various domains [32–34]. In the gaming world, DRL has achieved groundbreaking milestones, such as mastering the strategic complexity of StarCraft and surpassing human performance in Go with AlphaGo [35–37]. DRL's capabilities extend far beyond games, as demonstrated by AlphaFold, an AI system that revolutionized structural biology by accurately predicting protein 3D structures, solving a critical challenge in the life sciences [38]. In robotics [39], DRL enables autonomous systems like drones, robotic arms, and self-driving vehicles to learn complex control strategies for navigation and interaction with their environments [40,41].

RL has emerged as a powerful tool in scientific research, driving innovative approaches and breakthroughs across multiple fields [42–47]. Ding et al. [48] applied DRL to delay feedback control methods, replacing static gains with adaptive, time-varying gains generated by the agent, thereby overcoming the limitations of traditional control strategies. Krylov et al. [49] used RL to suppress synchronization in neuronal populations, demonstrating its effectiveness in controlling collective oscillation modes in models such as globally coupled limit-cycle Bonhoeffer–van der Pol oscillators and the bursting Hindmarsh–Rose neuron model. Biferale et al. [50] employed RL to address Zermelo's navigation problem, optimizing vessel paths in turbulent flows. Using the actor–critic RL algorithm, they identified near-optimal strategies for both time-independent and chaotic flow conditions.

This paper focuses on applying DRL to achieve generalized synchronization between a slave chaotic system and a master system. We propose a novel control scheme in which an agent interacts with both systems, learning to optimize its strategy and maximize cumulative rewards. To our knowledge, the application of DRL to control generalized chaotic synchronization has not been explored yet. This approach effectively tackles the challenges of nonlinearity and unpredictability inherent in chaotic systems, providing a fresh perspective on controlling synchronization in complex dynamic environments.

The remainder of this paper is organized as follows: Section 2 introduces the DRL method. In Section 3, we apply this method to typical chaotic systems, demonstrating generalized chaotic synchronization between the slave and master systems. We also show that the method can train a single agent capable of controlling generalized chaotic synchronization across multiple chaotic system configurations. Additionally, we demonstrate the method's effectiveness in environments with incomplete observable information and when the chaotic systems transition to high-dimensional systems. Finally, Section 4 provides a summary and concludes the paper.

2. Deep reinforcement learning (DRL) method

To address the challenge of achieving generalized synchronization between two chaotic systems, we propose a novel, model-free DRL strategy. Our approach involves two systems: a slave system (denoted as S) and a master system (M), each with its own n -dimensional state vector, $\mathbf{O}_S(t)$ and $\mathbf{O}_M(t)$. These vectors represent the measurable variables that govern the dynamics of the systems.

As shown in Fig. 1, the DRL setup consists of two main components: the environment, which includes both the slave and master systems, and an agent tasked with aligning the behavior of the slave system with that of the master system. At each discrete time step t , the agent observes the current state of the environment, $s(t)$, and selects an action $a(t)$ based on its policy π . This action drives the environment to a new state $s(t+1)$, and the agent receives a reward $r(t)$ that guides its learning process. In our framework, the environment's state, $s(t)$, is defined by the concatenated state vectors $\mathbf{O}_S(t)$ and $\mathbf{O}_M(t)$. The agent's action is represented by a compensation vector $\mathbf{A}(t)$, which is applied to the slave system. At time t , the slave system evolves naturally over a small time interval δt to produce $\mathbf{O}'_S(t+1)$. The application of $\mathbf{A}(t)$ updates the slave system's state variables to

$$\mathbf{O}_S(t+1) = \mathbf{O}'_S(t+1) + \mathbf{A}(t). \quad (1)$$

The action $\mathbf{A}(t)$ is determined by the interplay of two factors. The first is the difference between the state variables of the slave and master systems, $\Delta\mathbf{O}(t) = \mathbf{O}_S(t) - \mathbf{O}_M(t)$, which needs to be minimized to achieve synchronization. The second is the output of the policy network, the compensation vector $\mathbf{U}$. The agent evaluates the states $\mathbf{O}_S(t)$ and $\mathbf{O}_M(t)$, combining them into a single vector $(\mathbf{O}_S(t), \mathbf{O}_M(t))$, which is then fed into the policy network. This network produces $\mathbf{U}(t)$, an $n \times 1$ vector:

$$\mathbf{U}(t) = \begin{bmatrix} U_1 \\ U_2 \\ \dots \\ U_n \end{bmatrix}. \quad (2)$$

The action $\mathbf{A}(t)$ is then determined by:

$$\mathbf{A}(t) = \mathbf{U}(t) \cdot \Delta\mathbf{O}(t), \quad (3)$$

where $\cdot$ denotes the element-wise product. Each element U_i represents the corrective action applied to the i th variable of the slave system, scaled by the corresponding element of $\Delta\mathbf{O}(t)$. The vector $\Delta\mathbf{O}(t)$ ensures that the agent's actions are adapted to the current training context, thereby reducing the training cost and accelerating convergence.

The reward function $r(t)$, which is crucial for guiding the agent's actions, consists of two components: an n -dimensional reward vector $\mathbf{r}_1$ and an n -dimensional penalty vector $\mathbf{r}_2$. We introduce three n -dimensional threshold vectors, $\mathbf{C}_1$, $\mathbf{C}_2$, and $\mathbf{C}_3$, with the hierarchy $C_{3i} < C_{1i} < C_{2i}$ for all components. The training process is divided into two phases:

- **Phase I:** The agent is trained under relaxed constraints defined by $\mathbf{C}_1$ and $\mathbf{C}_2$, allowing it to learn effective control strategies.
- **Phase II:** The constraints are strengthened by incorporating $\mathbf{C}_3$ and $\mathbf{C}_2$, prompting the agent to minimize synchronization error and improve control effectiveness.

The agent receives a reward of $r_{1i} = 0.5$ if $|\Delta O_i(t+1)|$ falls below C_{1i} (or C_{3i} in Phase II). The agent's reward is determined by the n -dimensional vector $\mathbf{D}(t)$:

$$\mathbf{D}(t) = \mathbf{O}_M(t+1) - \mathbf{O}'_S(t+1). \quad (4)$$

If $D_i(t)U_i(t) > 0$, the agent receives a reward of $r_{1i} = 0.1$; otherwise, the reward is zero. For each component of $\Delta\mathbf{O}(t)$, the agent incurs a

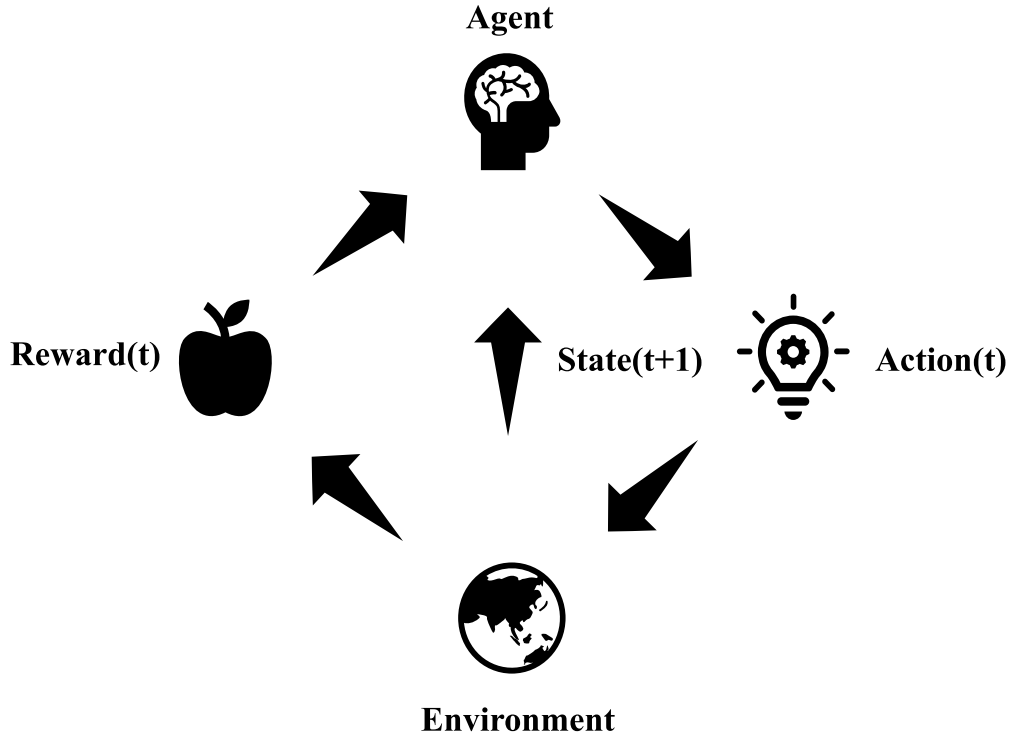


Fig. 1. A RL algorithm comprises two key components: the environment and the agent. At each time step t , the agent observes the environment's current state, $s(t)$, and executes an action, $a(t)$, based on its policy. This action causes the environment to transition to a new state, $s(t+1)$, and provides the agent with a reward, $r(t)$, which serves as feedback to guide its learning process.

penalty of $r_{2i} = -50$ when $|\Delta O_i(t+1)| > C_{2i}$, which may terminate the training episode. The total reward function r_t is defined as:

$$r_t = \sum_{i=1}^n (r_{1i} + r_{2i}) \quad \text{if } A_i(t) \neq 0, \quad (5)$$

where

$$r_{1i} = \begin{cases} 0.5, & \text{if } |\Delta O_i(t+1)| < C_{1i} \text{ (or } C_{3i} \text{ in Phase II)} \\ 0.1, & \text{if } |\Delta O_i(t+1)| \geq C_{1i} \text{ (or } C_{3i} \text{ in Phase II)} \\ & \text{and } D_i(t)U_i(t) > 0 \\ 0, & \text{if } |\Delta O_i(t+1)| \geq C_{1i} \text{ (or } C_{3i} \text{ in Phase II)} \\ & \text{and } D_i(t)U_i(t) \leq 0 \end{cases} \quad (6)$$

and

$$r_{2i} = \begin{cases} -50, & \text{if } |\Delta O_i(t+1)| > C_{2i} \\ 0, & \text{if } |\Delta O_i(t+1)| \leq C_{2i}. \end{cases} \quad (7)$$

The policy function $\pi(s(t), a(t))$ represents the probability that the agent will choose action $a(t)$ given the current state $s(t)$, which includes the states of both the slave and master systems. The primary objective of RL is to maximize the agent's cumulative rewards over time. The return, $G(s_t)$, represents the discounted sum of future rewards starting from state $s(t)$. The state value function, $V^\pi(s)$, indicates the expected return $G(s_t)$ when following a specific policy π , and is mathematically expressed as:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [G(a_t | s_t = s)] = \mathbb{E}_\pi \left[\sum_{k=1}^{\infty} \gamma^k r_{t+k} \mid s(t) = s \right], \quad (8)$$

where γ is the discount factor, with $0 < \gamma < 1$, which determines the relative importance of future rewards compared to immediate rewards. The agent is trained to maximize $V^\pi(s)$, with the goal of improving decision-making from a long-term perspective.

In our DRL application, both the policy function π and the state value function $V^\pi(s)$ are approximated using neural networks. Specifically, the policy function is represented as $a(t) = \pi(s(t) | \omega)$, and the state value function as $V^\pi(s) = v(s(t) | \theta)$, where ω and θ are the parameters of the respective neural networks [51–53].

The policy network and the value network are iteratively updated to approximate the optimal policy and state value functions. To train the agent, we use the Proximal Policy Optimization (PPO) algorithm [54], which is known for its robust stability and efficiency in environments with continuous action spaces, making it an ideal choice for our application.

3. Results

To evaluate the performance of our DRL method in achieving generalized chaotic synchronization, we test it on several well-known dynamical systems, including the Lorenz system, the Rössler system, and the Chua circuit. For these systems, the compensation vector $U(t)$ is composed of three components: U_1 , U_2 , and U_3 . To further demonstrate the versatility of our approach, especially for systems that are more susceptible to external disturbances and thus harder to synchronize, we also conduct tests on six-dimensional systems—coupled versions of the Lorenz, Rössler, and Chua systems. In these higher-dimensional cases, the compensation vector $U(t)$ is extended to include six components, corresponding to the increased complexity of the systems.

Generally, a three-dimensional dynamical system is described as

$$\begin{aligned} \frac{dx}{dt} &= f(x, y, z), \\ \frac{dy}{dt} &= h(x, y, z), \\ \frac{dz}{dt} &= g(x, y, z). \end{aligned} \quad (9)$$

For the Lorenz oscillator [55], $f(x, y, z) = \sigma(y - x)$, $h(x, y, z) = \rho x - xz - y$, and $g(x, y, z) = xy - \beta z$ with $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$. For the Rössler oscillator [56], $f(x, y, z) = -y - x$, $h(x, y, z) = x + ay$, and $g(x, y, z) = b + z(x - c)$ with $a = 0.2$, $b = 0.2$, and $c = 5.7$. For the Chua circuit [57], $f(x, y, z) = \alpha(y - f(x))$, $h(x, y, z) = x - y + z$, and $g(x, y, z) = \beta y$ with $\alpha = 10$, $\beta = 1/0.056$, and $f(x) = bx + 0.5(a - b)(|x + 1| - |x - 1|)$ with $a = -1.44$ and $b = -0.72$. Throughout this work, we set $C_{1x} = C_{1y} = C_{1z} = 1$, $C_{3x} = C_{3y} = C_{3z} = 0.5$, and $C_{2x} = C_{2y} = C_{2z} = 50$.

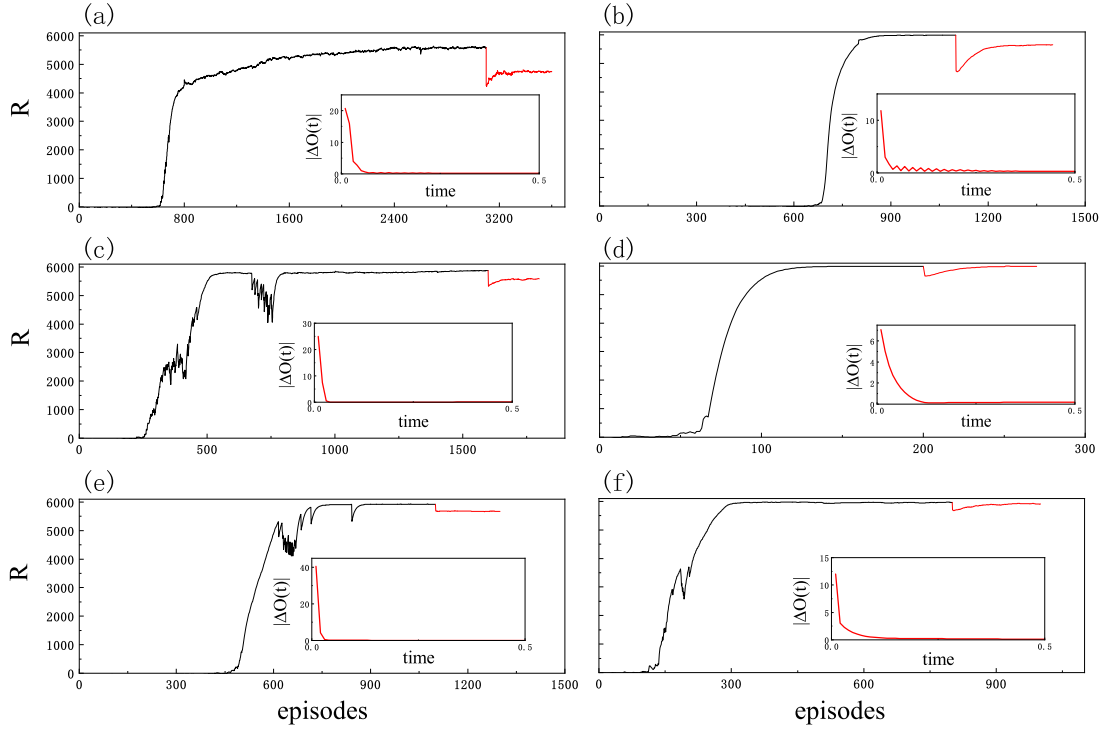


Fig. 2. The evolution of the accumulated reward R during the training for $L-R$ in (a), $L-C$ in (b), $R-L$ in (c), $R-C$ in (d), $C-L$ in (e) and $C-R$ in (f). The black solid line represents Phase I of training, and the red solid line represents Phase II. The insets of (a–f) show the synchronization error $|\Delta O(t)|$ against time. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

We first consider the scenarios where both the slave and master systems are three-dimensional. Six different system configurations are considered, Lorenz system as the slave one (S) and Rössler system as the master one (M) (denoted as $L-R$), Lorenz system (S) and Chua circuit (M) (denoted as $L-C$), Rössler system (S) and Lorenz system (M) (denoted as $R-L$), Rössler system (S) and Chua circuit (M) (denoted as $R-C$), Chua circuit (S) and Lorenz system (M) (denoted as $C-L$), and Chua circuit (S) and Rössler system (M) (denoted as $C-R$). In these scenarios, $O_S(t)$ and $O'_S(t)$ can be numerically integrated using the fourth-order Runge–Kutta method with a time step of $\delta t = 0.01$ time units.

The number of training episodes varies across different configurations, with each episode beginning with randomly initialized conditions and consisting of 4000 time steps. At each time step, the agent evaluates the state vectors $O_S(t)$ and $O_M(t)$ of the environment, computes the compensation vector $U(t)$, and applies the action $A(t)$ to $O'_S(t+1)$. This results in a new state vector $O_S(t+1)$ for the slave system and a corresponding reward $r(t)$. The slave system then evolves based on this updated state vector, which serves as the starting point for the next time step, δt . This process is repeated throughout each episode, with the agent adjusting the policy network parameters ω to maximize the cumulative reward $R = \sum_{t=1}^{4000} r_t$. Training is considered successful if the cumulative reward R approaches its maximum potential value of 6000 and stabilizes; otherwise, the training is deemed unsuccessful. After successful training, the agent is used to control the slave system, aiming to synchronize it with the master system. During testing, we monitor the evolution of $|\Delta O(t)|$, and generalized synchronization is confirmed when $|\Delta O(t)|$ approaches zero.

The training results of the agent are presented in Fig. 2. Focusing on the $L-R$ configuration, Fig. 2(a) illustrates the evolution of the accumulated reward R throughout the training process. In the initial phase, spanning the first 600 episodes, R remains near zero as the agent is in an exploratory stage, randomly searching for effective strategies. During this period, the absolute error of an individual state variable, $|\Delta O_i(t)|$, frequently exceeds the predefined upper limit C_{2i} , leading

to the premature termination of episodes. This phase is crucial for the agent to gradually accumulate a rich set of experiences. Between episodes 600 and 900, R rises sharply, indicating that the agent is beginning to develop more effective control strategies, resulting in fewer early terminations and an accelerated learning process. After episode 900, the growth of R slows down and eventually stabilizes near the maximum possible accumulated reward. By episode 3100, the agent completes Phase I of the training and moves on to Phase II. In Phase II, R converges rapidly. Notably, the maximum value of R in Phase II is lower than that in Phase I, due to the intensified constraints that make earning rewards more challenging, reflecting the need for more precise control strategies. However, the improvement in R during Phase II suggests that the neural network had not yet reached its full potential under the relaxed conditions of Phase I, and the stricter conditions of Phase II may further refine its strategies.

The trained agent is then tasked with controlling the chaotic generalized synchronization between the slave and master systems. The inset in Fig. 2(a) shows the synchronization error $|\Delta O(t)|$ over time. At the start of the control process, the initial synchronization error exceeds 20. However, as time progresses, $|\Delta O(t)|$ quickly approaches zero, indicating that chaotic generalized synchronization between the slave and master systems is achieved within a short period. In essence, the rapid rise of R to its peak value, followed by the agent's performance, demonstrates the effectiveness of our DRL method in managing chaotic generalized synchronization for the $L-R$ configuration.

For the $L-C$ configuration, Fig. 2(b) shows that the accumulated reward R rises quickly after 700 episodes and reaches a stable plateau at a significantly high value within the next 200 episodes. Once the trained agent is applied to the $L-C$ configuration, the synchronization error exhibits an oscillatory decline toward zero, following an initial steep drop. This behavior contrasts with the $L-R$ configuration, where the synchronization error decreases more monotonically. In the $R-L$ configuration, Fig. 2(c) shows that the accumulated reward R converges to its peak value after experiencing two pronounced oscillations. Despite these intense fluctuations during the training phase, the agent

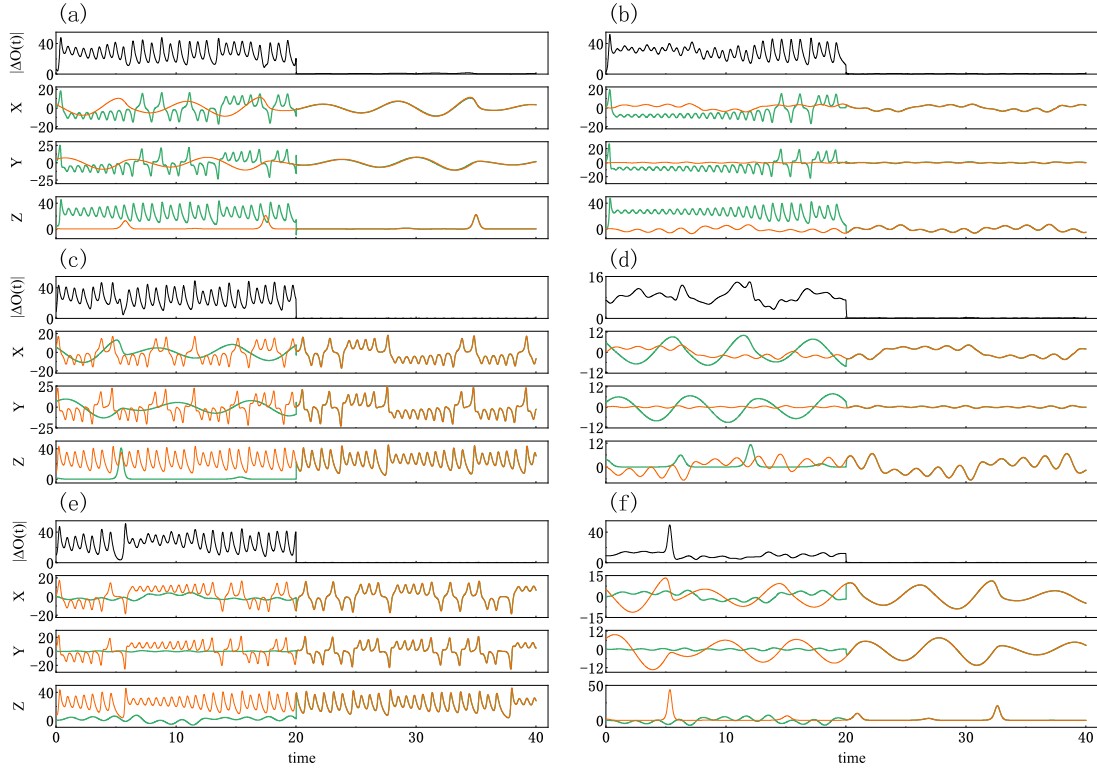


Fig. 3. Chaotic generalized synchronization dynamics and synchronization errors $|\Delta O(t)|$ of (a) $L-R$, (b) $L-C$, (c) $R-L$, (d) $R-C$, (e) $C-L$ and (f) $C-R$, with the control is activated at time = 20. The green solid line represents the trajectory of the slave system, and the red solid line represents the trajectory of the master system. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

retains exceptional control over chaotic generalized synchronization. Fig. 2(d), (e), and (f) demonstrate that the DRL approach is also effective for the $R-C$, $C-L$, and $C-R$ configurations. Notably, in the $C-L$ configuration during Phase II of training, the accumulated reward R remains nearly constant, suggesting that the neural network had already achieved optimal performance by the end of Phase I. Across all configurations, our DRL method successfully achieves generalized synchronization between the slave and master systems within a quarter of a time unit. This highlights the robustness and efficiency of our method in controlling synchronization across different system configurations.

To be noted, our training and testing are based on random initial conditions, so variations in the initial conditions do not affect the stability of the control strategy. We have examined the synchronization error for the DRL control strategy under the influence of noise and found it can be maintained at the 10^{-2} level, demonstrating that the DRL method has good robustness.

Fig. 3 provides a more intuitive view of the slave system's trajectory shifts before and after the agent's control is activated, offering a long-term dynamic perspective. Specifically, Fig. 3(a) tracks the dynamics of the slave and master systems under the $L-R$ configuration using the synchronization error $|\Delta O(t)|$ and the trajectories of the X , Y , and Z variables. The green solid line represents the trajectory of the slave system, while the red solid line represents that of the master system. Once both the slave and master systems reach their steady states, the agent initiates control at time $t = 20$. Upon activation, the trajectories of the slave system's three variables nearly instantaneously align with those of the master system. Moreover, for the next 20 time units, the synchronization between the two systems remains highly stable. Notably, even though the Lorenz system's period differs significantly from that of the Rössler system, the effectiveness of the DRL method remains unaffected. Similarly, as shown in other configuration cases in Fig. 3, our method can efficiently and stably achieve generalized synchronization.

We compare the DRL method with the traditional linear feedback approach [58], where the gain coefficient for the linear feedback method is set to 80. As shown in Fig. 4(a1) where $L-R$ configuration is used, the DRL method outperforms the linear feedback approach in terms of reducing and stabilizing the synchronization error. Analogous observations are also evident in the remaining five configurations. In addition to control efficiency and stability, energy cost [59] is also an important performance indicator for chaos synchronization control methods. Fig. 4(b) compares the energy cost of the DRL method and the linear feedback method across six configurations. It is clear that the DRL method results in lower energy cost compared to the linear feedback approach.

To further validate the effectiveness of our DRL method, we aim to incorporate all six configurations into a single training session to determine whether a single trained agent can efficiently manage the control strategies for all configurations simultaneously. In Phase I of the training, we set $\delta t = 0.01$, and in Phase II, we refined it to $\delta t = 0.001$. The shorter time interval in Phase II allows the agent to accumulate more experiences while maintaining a constant episode length, thereby improving training efficiency. In Phase I, we follow the original sequence of configurations, but for Phase II, we strategically reorder them to $L-C$, $L-R$, $R-L$, $R-C$, $C-L$, and $C-R$ to enhance training efficiency. It is important to note that while training can succeed with any sequence, the efficiency may vary. We have selected the sequence that we believe offers the most efficient training process.

Fig. 5(a) shows the progression of the accumulated reward R as the agent is trained sequentially across all six configurations. The R graph reveals a pattern of distinct segments and plateaus. During the first 3600 episodes with the $L-R$ configuration, R begins to rise sharply around the 600th episode. After approximately 1000 episodes, the rate of increase slows down, gradually approaching the maximum reward value, with R eventually stabilizing at this peak. After 3600 episodes, the configuration switches to $L-C$, triggering another rapid

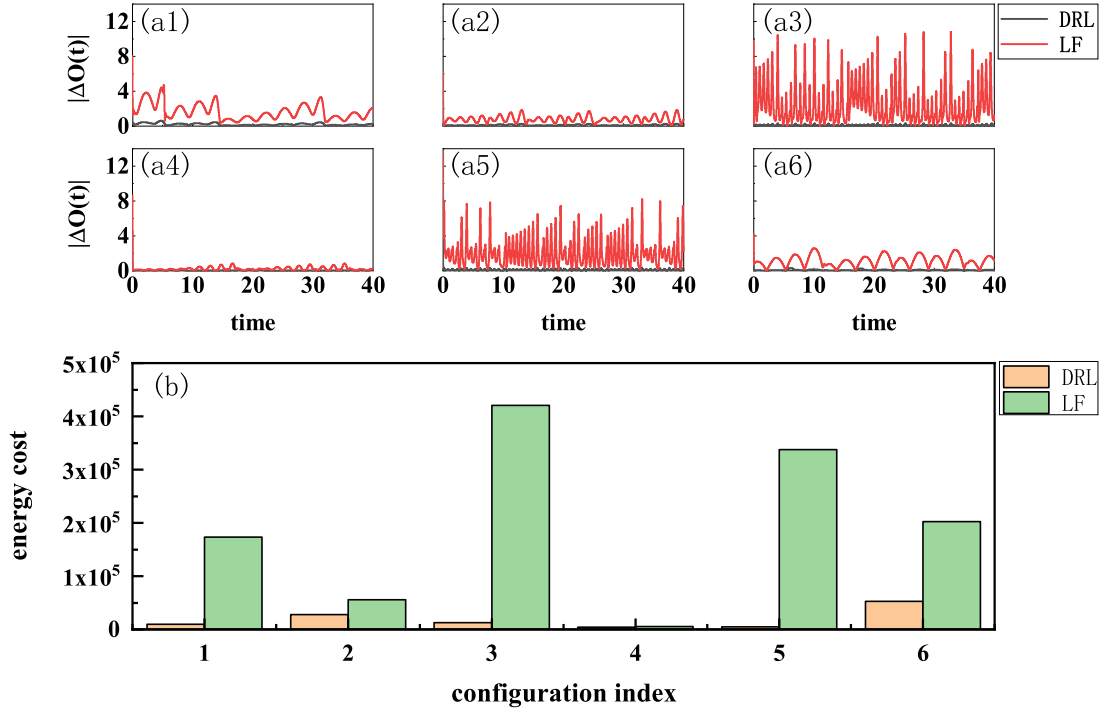


Fig. 4. Comparison of the DRL Method and the linear feedback Method. (a1-a6) Synchronization error $|\Delta O(t)|$ curves when using the DRL method and the linear feedback method under different configurations. (a1) $L-R$, (a2) $L-C$, (a3) $R-L$, (a4) $R-C$, (a5) $C-L$, (a6) $C-R$. The black solid line represents the DRL method, while the red solid line represents the linear feedback method. (b) A comparison of the energy cost between the DRL method and the linear feedback method under different configurations. The configuration indices 1 to 6 denote $L-R$, $L-C$, $R-L$, $R-C$, $C-L$, and $C-R$, respectively. Orange represents the DRL method, while green represents the linear feedback method. The testing duration is 10. The gain coefficient of linear feedback control is set to 80. $\delta t = 0.001$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

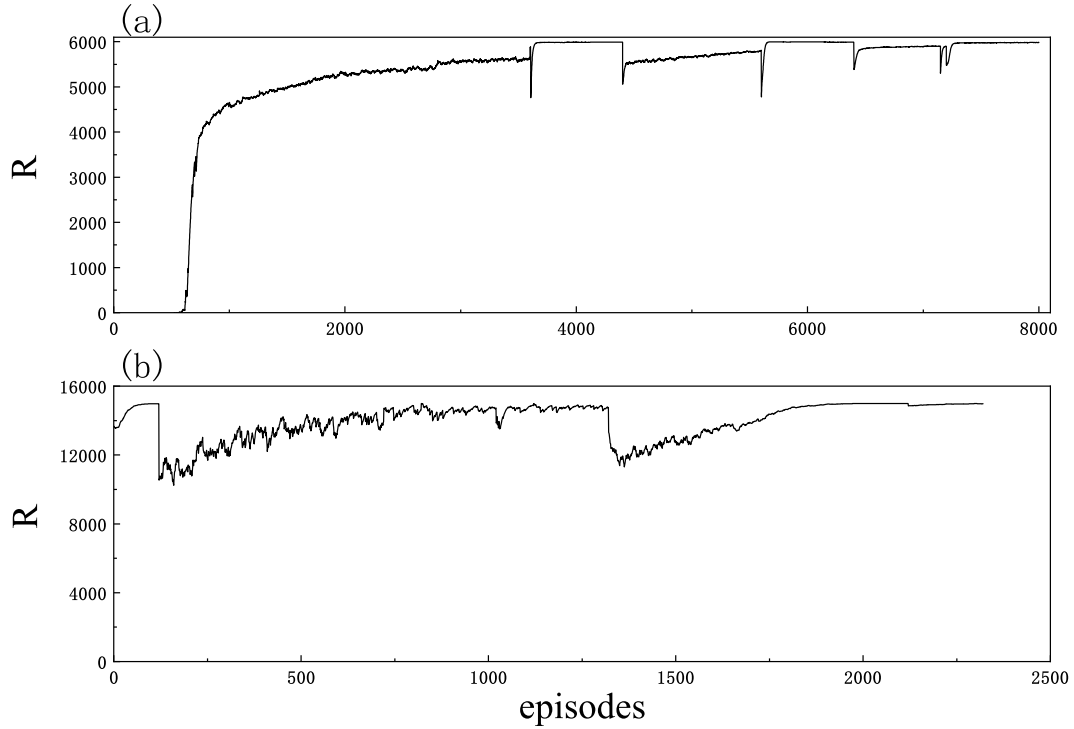


Fig. 5. The evolution of the accumulated reward R during the training for six configurations. (a) represents Phase I ($\delta t = 0.01$) of the training. (b) shows Phase II ($\delta t = 0.001$), with the maximum reward value being 16000.

increase in R , which then levels off into a plateau. At episode 4400, the environment transitions to the $R-L$ configuration, where R continues to climb steadily, albeit more slowly, before stabilizing at the maximum

reward value. This trend repeats as the environment moves to the $R-C$, $C-L$, and $C-R$ configurations at episodes 5600, 6400, and 7200, respectively, each transition resulting in a plateau in the R curve. These

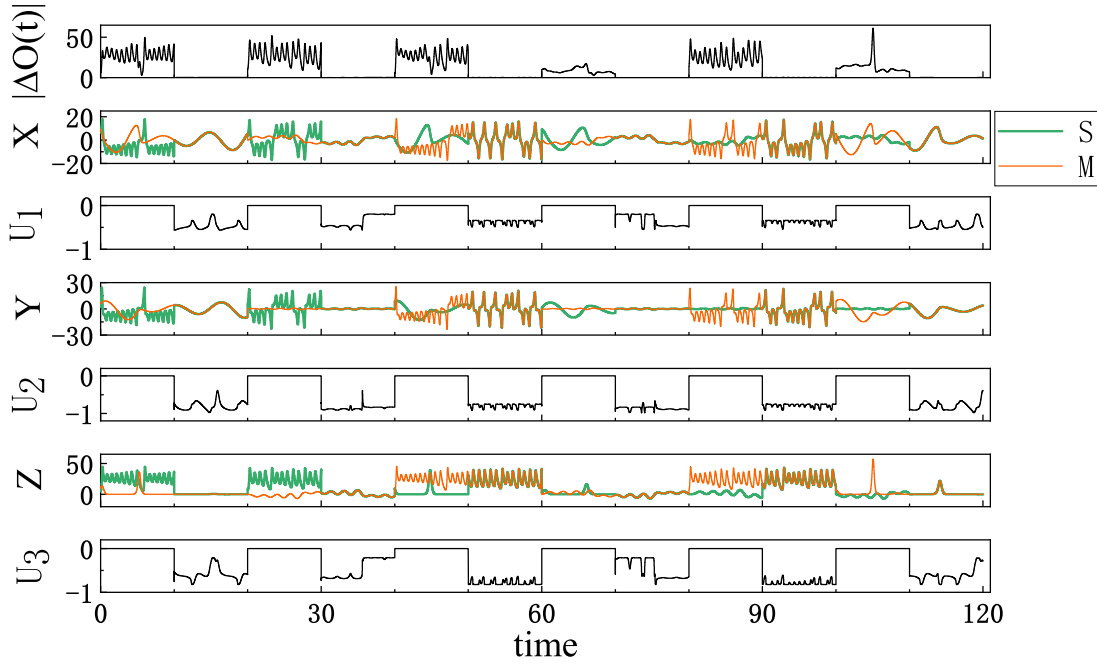


Fig. 6. The evolution of the chaotic generalized synchronization dynamics of the slave and the master systems, compensation vector (U_1, U_2, U_3) and synchronization errors $|\Delta O(t)|$ over 120 time units, where the configuration from time units 0 to 19 is $L-R$, from 20 to 39 is $L-C$, from 40 to 59 is $R-L$, from 60 to 79 is $R-C$, from 80 to 99 is $C-L$ and from 100 to 119 is $C-R$, with the control is activated within the time periods 10–19, 30–39, 50–59, 70–79, 90–99 and 110–119. The green solid line represents the trajectory of the slave system, and the red solid line represents the trajectory of the master system. Each time the configuration is switched, the initial conditions are randomly initialized. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

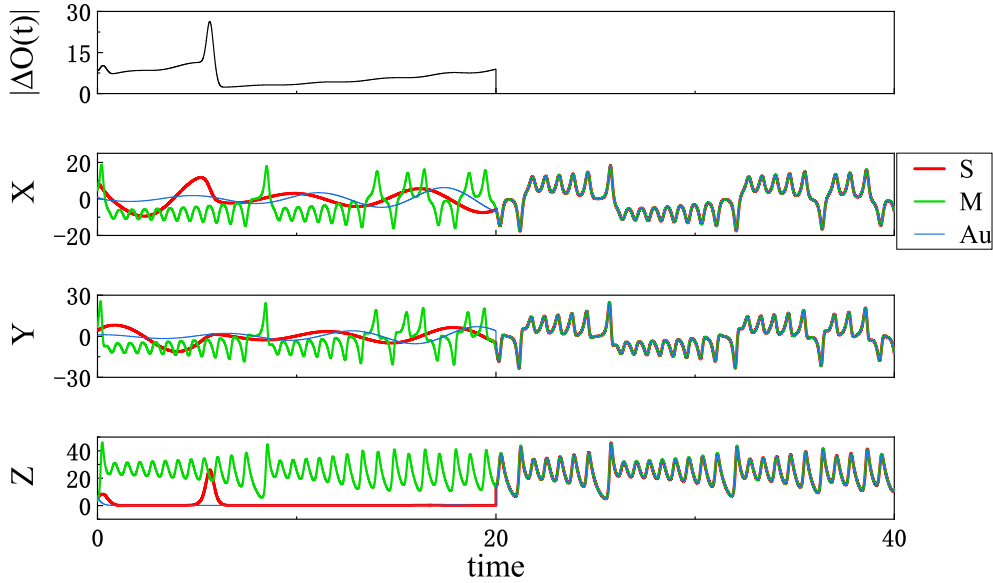


Fig. 7. The evolution of chaotic generalized synchronization dynamics and the error $|\Delta O(t)|$ between the slave system and the auxiliary slave system in $R-L$ configuration, with the control of the agent which is identical to that used in Fig. 6 is activated at time = 20. The green, red and blue solid line represent the trajectory of the master system, the slave system and the auxiliary slave system separately. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

plateaus occur because the agent, having been trained on previous configurations, is able to quickly adapt to the new environments. The accumulated reward R converges and stabilizes near the maximum value within just a few episodes. This adaptability highlights the neural network's generalization capabilities, which is a key advantage of our approach.

Fig. 5(b) illustrates the progression of the accumulated reward R in Phase II. Building upon the foundation established in Phase I, Phase II requires fewer episodes to reach the maximum reward values. Under the $L-C$ configuration, R achieves its maximum accumulated value within just 120 episodes. When the configuration switches to $L-R$,

R fluctuates and increases over 1100 episodes before stabilizing near the maximum value. The following 900 episodes are under the $R-L$ configuration. Notably, when the $R-C$ configuration is introduced, the accumulated reward reaches its maximum value within a single episode, indicating that no additional training is needed for the $R-C$ configuration in Phase II. Our tests suggest that the agent has already attained optimal performance with the $R-C$ configuration by Phase II, making further training unnecessary. Similarly, after 200 episodes with the $C-L$ configuration, the agent achieves peak performance in Phase II with the $C-R$ configuration, again eliminating the need for further training.

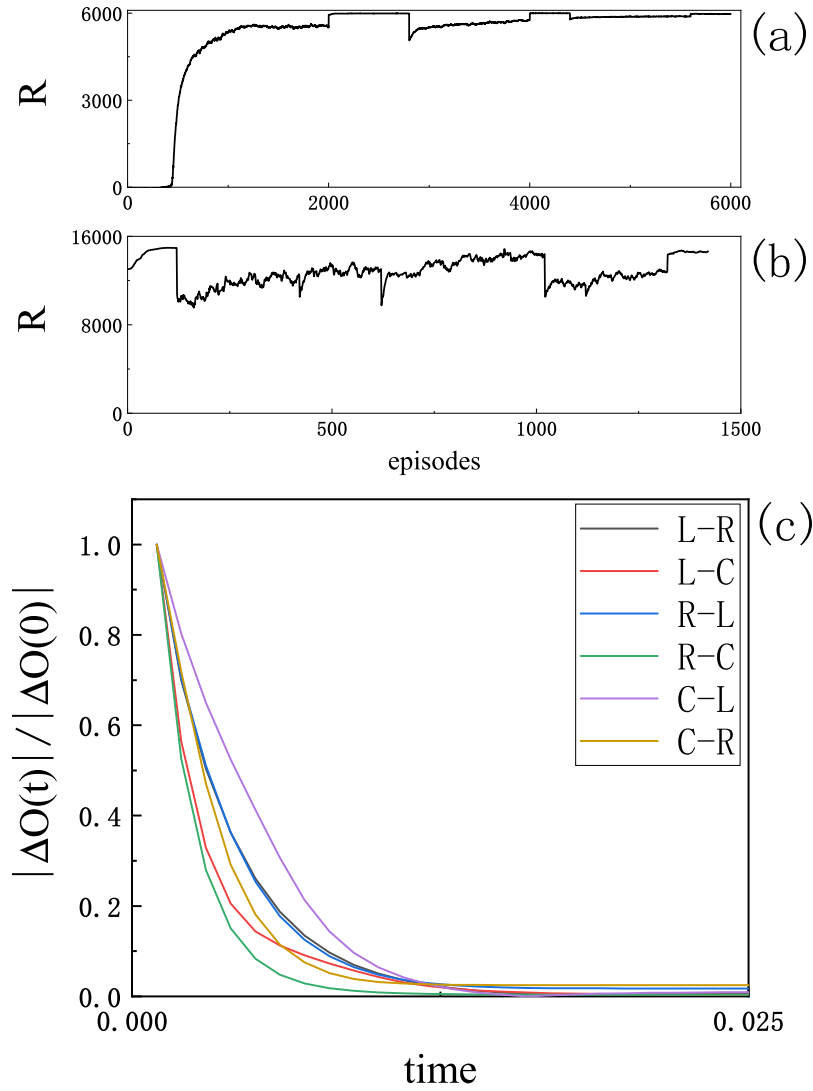


Fig. 8. Evolution of the cumulative reward R and normalized synchronization error when the observable state is $\mathbf{o}(t) = (x_S, y_S, x_M, y_M)$ and the agent is trained with successive six configurations. (a) illustrates the initial phase of training (Phase I) with a time step $\delta t = 0.01$. (b) depicts the refined phase of training (Phase II) with a decreased time step $\delta t = 0.001$ targeting a maximum reward value of 16 000. (c) presents the normalized synchronization error $\frac{|\Delta \mathbf{o}(t)|}{|\Delta \mathbf{o}(0)|}$ across the six system configurations, showcasing the agent's performance post-training.

Next, we evaluate the performance of the trained agent across all six configurations. Fig. 6 shows the dynamics of both the slave and master systems over a period of 120 time units, along with the three components of the compensation vector (U_1 , U_2 , and U_3) and the synchronization error $|\Delta \mathbf{o}(t)|$. During the first 19 time units, control is initiated by the agent at $t = 10$ and sustained for 10 time units. The synchronization error drops dramatically from a value above 25 to nearly zero, indicating that the agent's performance is comparable to that of an agent trained specifically for the $L - R$ configuration. Throughout the 10–19 time period, U_1 , U_2 , and U_3 are actively engaged, each displaying distinct waveform characteristics. U_1 shows three consecutive peaks, slightly lagging behind the peaks and valleys of the X variable's waveform. Similarly, U_2 's waveform mirrors this pattern, with three peaks aligning with the Y variable's waveform. In contrast, U_3 exhibits a valley-peak-valley pattern, distinct from the waveforms of U_1 and U_2 . The Z variable's waveform remains stable and nearly flat during the 10–19 time period, with no corresponding fluctuations in U_3 's waveform.

At $t = 20$, the configuration transitions to $L - C$, with control reinitiated by the agent at $t = 30$. The agent maintains a quick control response and stable control effects. The waveforms of U_1 and U_3 display

step-like features, while U_2 presents a single pulse. The step-like progression in U_1 's waveform, from low to high, mirrors the Chua circuit's oscillation between its two intertwined chaotic attractors. Similarly, the Z variable's waveform exhibits step-like features, likely due to the attractor transition. The step-like feature in U_3 's waveform suggests a low-to-high progression, indicating that as the Chua circuit oscillator moves between attractors, the agent must apply weaker control to the slave system, which is reflected in the pulse seen in U_2 's waveform. However, a notable difference between U_2 and U_1 , U_3 is that post-attractor transition, the X and Z variables of the slave system require sustained, weaker control from the agent, while the control intensity for the Y variable remains consistent with pre-attractor transition levels.

The agent consistently demonstrates strong control performance across the subsequent $R - L$, $R - C$, $C - L$, and $C - R$ configurations, exhibiting the same level of proficiency as an agent trained on a single configuration. In the $R - L$ configuration, U_1 and U_2 display waveforms with distinct valley patterns, while U_3 presents a dense peak pattern. In the $C - L$ configuration, the waveforms of U_1 , U_2 , and U_3 closely resemble those seen in the $R - L$ configuration. In the $R - C$ configuration, U_1 and U_3 continue to show step-like waveforms, reflecting the dynamic shifts of the Chua circuit oscillator as it transitions between attractors. Notably, the waveform steps begin at a high point and decrease in

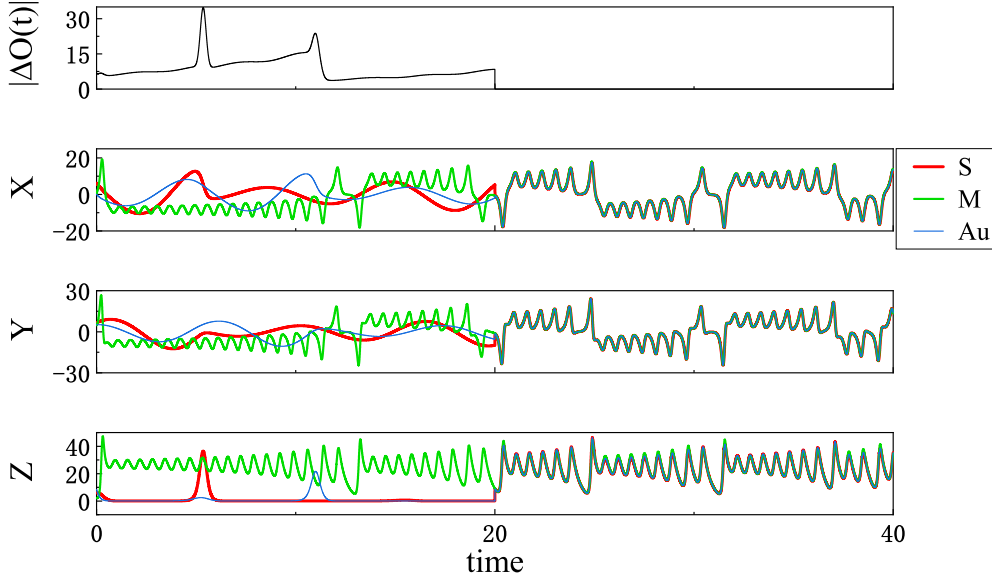


Fig. 9. The evolution of chaotic generalized synchronization dynamics and the error $|\Delta O(t)|$ between the slave system and the auxiliary slave system in $R-L$ configuration, with the control of the agent which is identical to that used in Fig. 8 is activated at time = 20. The green, red and blue solid line represent the trajectory of the master system, the slave system and the auxiliary slave system separately. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

height, indicating that the agent must initially apply stronger control as the oscillator moves to a new attractor. This requirement contrasts with that in the $R-C$ configuration. Additionally, before the attractor transition, both U_1 and U_2 exhibit dual valleys in their waveforms, a feature not observed in the $R-C$ configuration. These variations can be attributed to the differences in the dynamics of the slave systems. In the $L-R$ and $C-R$ configurations, the waveforms of U_1 , U_2 , and U_3 show similar but distinct characteristics.

From this analysis, it is clear that a single agent trained with our DRL method can efficiently achieve generalized chaotic synchronization control across different slave and master system configurations. Moreover, we observe that the waveform characteristics of U_1 , U_2 , and U_3 vary depending on the master system, highlighting the significant influence of the master system on these waveforms. While the differences in the slave system also affect the waveforms, their impact is less pronounced than that of the master system. This underscores the versatility and adaptability of our DRL approach in managing complex synchronization tasks across various system configurations.

To validate that the synchronization observed between the master and slave systems in Fig. 6 represents generalized synchronization, we apply the auxiliary system method [60]. An auxiliary slave system (denoted as Au), identical to the original slave system, is introduced. To ensure differentiation, the auxiliary slave system and the original slave system are initialized with distinct initial conditions. The master, slave, and auxiliary slave systems evolve naturally for 20 time units before the agent's control is activated. At time $t = 20$, the agent begins interacting with the system. The agent interacts exclusively with the master-slave system environment, while the auxiliary slave system remains external to this environment. Despite this, both the slave and auxiliary slave systems receive identical control signals from the agent.

Fig. 7 illustrates the dynamics of the master, slave, and auxiliary slave systems, as well as the evolution of the error $|\Delta O(t)|$ between the slave and auxiliary slave systems. Upon activation of the agent's control, the error $|\Delta O(t)|$ quickly converges to zero, indicating complete synchronization of the trajectories between the slave and auxiliary slave systems. This outcome confirms that the synchronization observed in Fig. 6 represents generalized synchronization.

The above results assume that the trained agent has access to all state variables of both the slave and master systems independently.

However, in real-world scenarios, it is often the case that only partial information is available for training. For example, only some of the state variables may be accessible. To address this, we further evaluate the performance of the DRL method under conditions of incomplete information, specifically focusing on how a single agent can achieve chaotic generalized synchronization between the slave and master systems across all six configurations simultaneously.

First, we consider a scenario where the agent can observe only two variables from each system, resulting in a state vector $\mathbf{o}(t) = (x_S, y_S, x_M, y_M)$. As shown in Fig. 8, the training results under these conditions are quite intriguing. Fig. 8(a) illustrates the evolution of the accumulated reward R during Phase I of the training. Interestingly, compared to the complete information scenario in Fig. 5(a), the number of training episodes required when information is incomplete is fewer, which is counterintuitive. The number of training episodes for each configuration is as follows: 2000 ($L-R$), 800 ($L-C$), 1200 ($R-L$), 400 ($R-C$), 1200 ($C-L$), and 400 ($C-R$). The accumulated reward R curve also shows multiple plateaus, indicating periods of stability during the learning process. Fig. 8(b) reveals that the total number of episodes in Phase II of the training is less than in Fig. 5(b), which corresponds to the complete information scenario. The distribution of training episodes in Phase II is as follows: 120 ($L-C$), 900 ($L-R$), 300 ($R-L$), 0 ($R-C$), 100 ($C-L$), and 0 ($C-R$). Notably, the agent does not require additional training in Phase II for the $R-C$ and $C-R$ configurations, as it already achieves optimal performance, consistent with the complete information scenario. Fig. 8(a) and (b) confirm that even with the partial state information $\mathbf{o}(t) = (x_S, y_S, x_M, y_M)$, the training is successful and requires fewer episodes than in the complete information scenario. Fig. 8(c) presents the normalized synchronization error curves for chaotic synchronization control across the six configurations, demonstrating the agent's ability to achieve synchronization within 0.025 time units for each configuration. This result highlights that the agent's control performance remains excellent, even with incomplete environmental observations, and is comparable to performance under full information conditions. Furthermore, the results in Fig. 8 confirm that the synchronization achieved between the master and slave systems represents generalized synchronization.

Next, we further reduce the available environmental information, limiting the agent's observations to only $\mathbf{o}(t) = (x_S, x_M)$. The outcomes of this experiment are shown in Fig. 10. The distribution of training

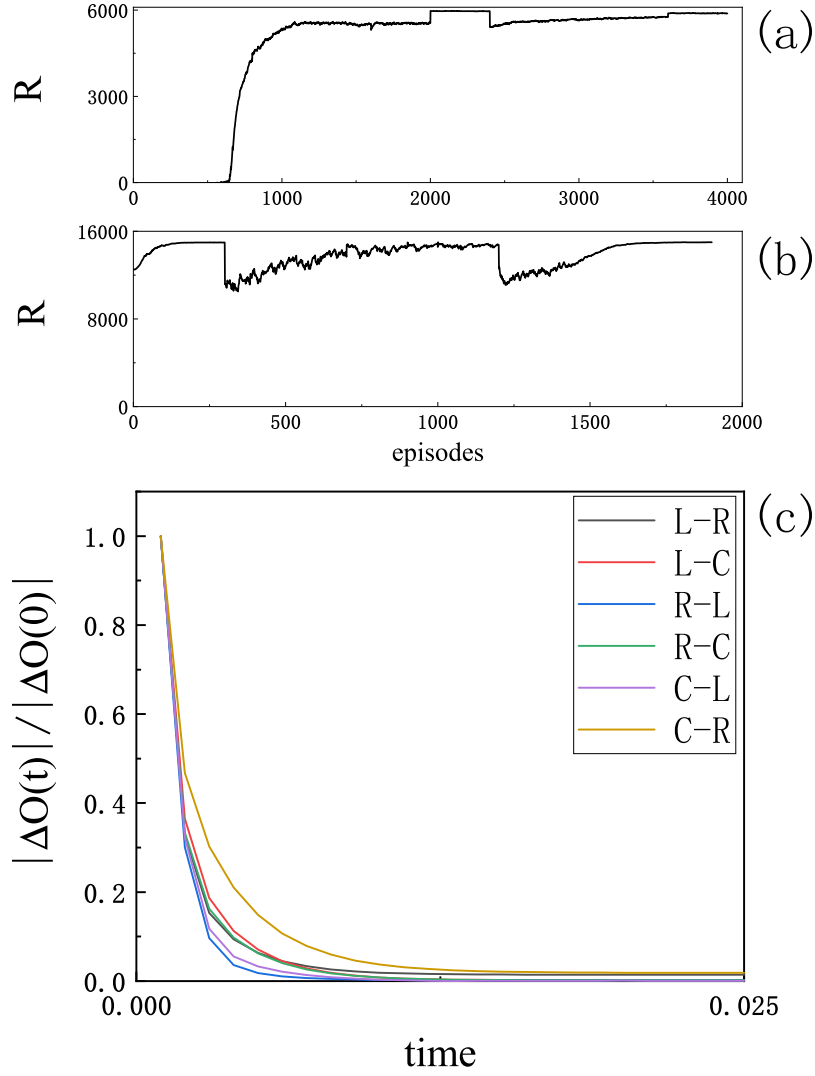


Fig. 10. Evolution of the cumulative reward R and normalized synchronization error when the observable state is $\mathbf{o}(t) = (x_S^1, x_M^1)$ and the agent is trained with successive six configurations. (a) illustrates the initial phase of training (Phase I) with a time step $\delta t = 0.01$. (b) depicts the refined phase of training (Phase II) with a decreased time step $\delta t = 0.001$ targeting a maximum reward value of 16000. (c) presents the normalized synchronization error $\frac{|\Delta \mathbf{O}(t)|}{|\Delta \mathbf{O}(0)|}$ across the six system configurations, showcasing the agent's performance post-training.

episodes for Phase I is as follows: 2000 ($L-R$), 400 ($L-C$), 1200 ($R-L$), 0 ($R-C$), 400 ($C-L$), and 0 ($C-R$). In Phase II, the distribution is as follows: 300 ($L-C$), 800 ($L-R$), 800 ($R-L$), 0 ($R-C$), 0 ($C-L$), and 0 ($C-R$). Interestingly, when the agent's observational capabilities are further restricted, the training requires even fewer episodes than when the agent had access to $\mathbf{o}(t) = (x_S, y_S, x_R, y_R)$. In Phase I, the agent achieves optimal control performance for the $R-C$ and $C-R$ configurations without requiring additional training in those specific environments. In Phase II, the agent requires no further training for the $R-C$, $C-R$, and $C-L$ configurations. Fig. 10(c) illustrates the agent's performance in controlling chaotic synchronization between the slave and master systems across all six configurations. The rapid convergence of the normalized synchronization error to near zero within 0.025 time units demonstrates the effectiveness of our method. Fig. 11 confirms that the synchronization achieved between the master and slave systems in Fig. 10 corresponds to generalized synchronization. From Figs. 8 to 11, it is clear that our DRL approach maintains robust performance even under incomplete information scenarios, and in some cases, it even outperforms the training efficiency observed when complete information is available.

Thus far, our findings have been based on the assumption that both the slave and master systems are three-dimensional. This naturally

leads to the question: does the DRL method remain effective when controlling high-dimensional chaotic systems? The answer is yes. To explore this, we create a six-dimensional coupled chaotic system by connecting two identical three-dimensional systems: the Lorenz system, the Rössler system, and the Chua circuit, with their evolution described as follows:

$$\begin{aligned}
 \frac{dx_1}{dt} &= f(x_1, y_1, z_1) + \mu(x_2 - x_1), \\
 \frac{dy_1}{dt} &= g(x_1, y_1, z_1), \\
 \frac{dz_1}{dt} &= h(x_1, y_1, z_1), \\
 \frac{dx_2}{dt} &= f(x_2, y_2, z_2) + \mu(x_1 - x_2), \\
 \frac{dy_2}{dt} &= g(x_2, y_2, z_2), \\
 \frac{dz_2}{dt} &= h(x_2, y_2, z_2).
 \end{aligned} \tag{10}$$

As the dimensionality increases to six, the compensation vector also expands to include six elements. In scenarios with complete information, the agent observes a state vector given by $\mathbf{s}(t) = (x_S^1, y_S^1, z_S^1, x_S^2, y_S^2, z_S^2)$,

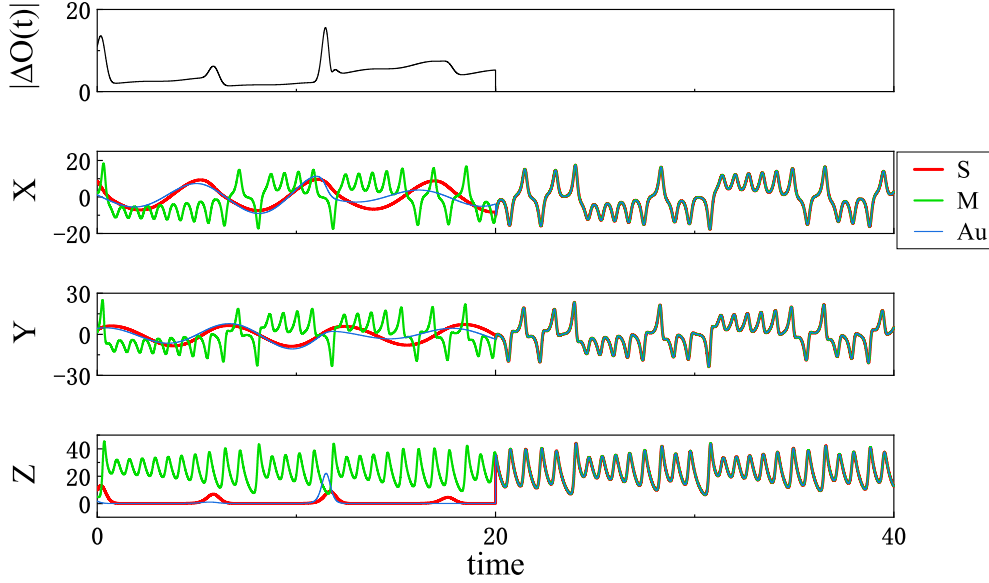


Fig. 11. The evolution of chaotic generalized synchronization dynamics and the error $|\Delta O(t)|$ between the slave system and the auxiliary slave system in $R-L$ configuration, with the control of the agent which is identical to that used in Fig. 10 is activated at time = 20. The green, red and blue solid line represent the trajectory of the master system, the slave system and the auxiliary slave system separately. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

$y_S^2, z_S^2, x_M^1, y_M^1, z_M^1, x_M^2, y_M^2, z_M^2$). We then test the DRL method under more challenging conditions, where the state observable by the agent is reduced to $\mathbf{o}(t) = (x_S^1, x_M^1)$. The results are shown in Fig. 12. The distribution of training episodes for Phase I is as follows: 2800 ($L-R$), 800 ($L-C$), 400 ($R-L$), 0 ($R-C$), 400 ($C-L$), and 0 ($C-R$). For Phase II, the distribution is: 150 ($L-C$), 800 ($L-R$), 700 ($R-L$), 0 ($R-C$), 0 ($C-L$), and 0 ($C-R$). The total number of training episodes is comparable to that in Fig. 10(a), where the chaotic system is three-dimensional, and the agent observes $\mathbf{o}(t) = (x_S, x_R)$. Notably, in Phase I for configurations $R-C$ and $C-R$, as well as in Phase II for configurations $R-C$, $C-L$, and $C-R$, the agent achieves optimal performance without additional training, mirroring the results from the three-dimensional system scenario.

Although high-dimensional chaotic systems are more susceptible to external disturbances, which increases the complexity of synchronization control, Fig. 12(c) demonstrates that the normalized synchronization error converges within 0.025 time units across all six configurations. These results confirm that even when tasked with controlling high-dimensional chaotic systems for generalized chaotic synchronization, the DRL method remains effective and robust.

We further test the chaotic synchronization between two six-oscillator coupled (18-dimensional) systems. The agent observes a randomly selected state vector $\mathbf{o}(t) = (z_S^1, y_M^3)$ with z_S^1 representing the z variable of the first oscillator in the slave system and y_M^3 representing the y variable of the third oscillator in the master system. The training configuration is set to $R-C$. As shown in Fig. 13, our DRL method is also capable of achieving synchronization between these high-dimensional systems, demonstrating its great scalability for practical applications.

4. Conclusion

In this paper, we have developed and demonstrated a model-free deep reinforcement learning (DRL) approach for achieving generalized chaotic synchronization across distinct chaotic systems. Our DRL approach has several advantages. Firstly, the method does not rely on prior knowledge of the systems' underlying dynamics and exhibits good performance across various classical chaotic systems, including the Lorenz system, the Rössler system, and the Chua circuit. Through continuous interaction with both the slave and master systems, the agent

optimizes its strategy to apply external compensation, thereby facilitating synchronization and maximizing cumulative rewards. Secondly, different from most reinforcement learning schemes for chaotic control where the design of the reward function is outcome-oriented, the reward function in our DRL approach is not only outcome-oriented but also process-guided, which increases the likelihood of neural network training convergence and significantly improves training efficiency. Thirdly, the approach adopts the PPO algorithm, which is known for its stability, making it particularly suitable for chaotic system control, a task that is highly sensitive to disturbances. Finally, in this approach, a single trained agent can generalize synchronization control across multiple slave-master system configurations, demonstrating the versatility and robustness even when the observable state information is limited.

Our findings highlight several key contributions and advantages of our DRL approach. First, the method's ability to achieve rapid synchronization in a remarkably short period represents a significant advancement in chaos control. Second, the versatility of our approach is demonstrated by its capacity to train a single agent capable of controlling generalized chaotic synchronization across multiple distinct slave-master system configurations. This flexibility is especially valuable for practical applications where system dynamics may vary.

Furthermore, the robustness of our method is emphasized by its effectiveness in maintaining synchronization control even with limited information about the state variables of the slave and master systems. This is particularly evident in the case of high-dimensional coupled chaotic systems, where the agent requires only minimal state information to successfully execute control.

In conclusion, our DRL approach provides a powerful and flexible tool for managing generalized chaotic synchronization. Its success across a range of system configurations and under conditions of incomplete information underscores its potential for wide-ranging applications in fields requiring precise control of complex dynamical systems. Future work will focus on further enhancing the method's efficiency and exploring its applicability in more complex systems and real-world scenarios.

CRediT authorship contribution statement

Haixin Cheng: Writing – original draft, Methodology, Investigation, Conceptualization. **Haihong Li:** Visualization, Software, Data curation. **Jinfeng Liang:** Methodology, Conceptualization. **Qionglin Dai:**

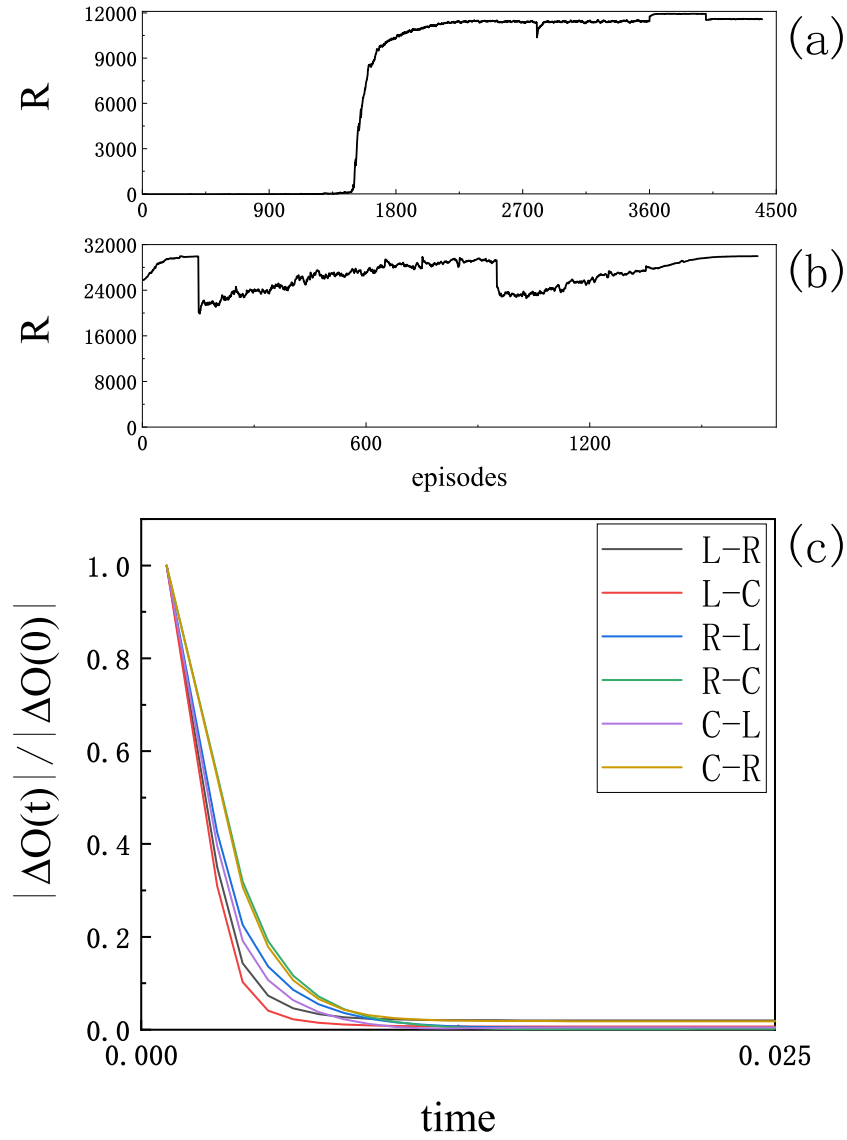


Fig. 12. Evolution of the cumulative reward R and normalized synchronization error for six-dimensional coupled chaotic systems when the observable state is $\mathbf{o}(t) = (x_S^1, x_M^1)$ and the agent is trained with successive six configurations. (a) illustrates the initial phase of training (Phase I) with a time step $\delta t = 0.01$. (b) depicts the refined phase of training (Phase II) with a decreased time step $\delta t = 0.001$ targeting a maximum reward value of 16 000. (c) presents the normalized synchronization error $\frac{|\Delta O(t)|}{|\Delta O(0)|}$ across the six system configurations, showcasing the agent's performance post-training.

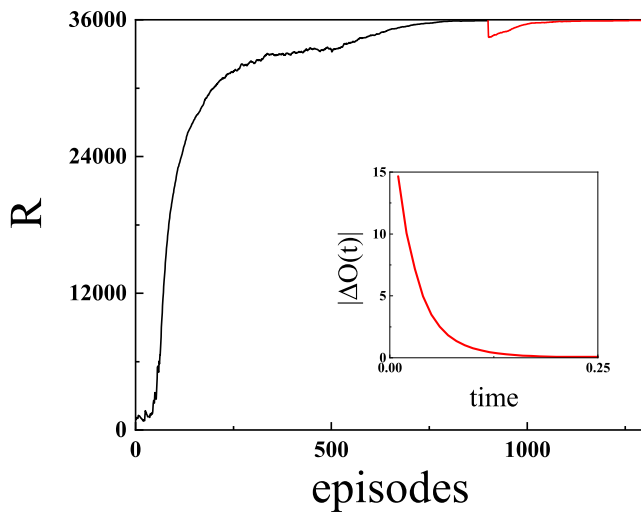


Fig. 13. The evolution of the accumulated reward R during the training phase for the $R-C$ configuration when the observable state is $\mathbf{o}(t) = (z_S^1, y_M^3)$. Both the master and slave systems are six-oscillator coupled chaotic systems. The black solid line represents Phase I, and the red solid line represents Phase II. The inset shows the synchronization error against time with $\delta t = 0.01$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Visualization, Validation. **Junzhong Yang:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] Pecora LM, Carroll TL. Synchronization in chaotic systems. *Phys Rev Lett* 1990;64:821.
- [2] Guan SG, Li K, Lai C-H. Chaotic synchronization through coupling strategies. *Chaos* 2006;16:023107.
- [3] Heagy JF, Carroll TL, Pecora LM. Synchronous chaos in coupled oscillator systems. *Phys Rev E* 1994;50:1874.
- [4] Feki M. An adaptive chaos synchronization scheme applied to secure communication. *Chaos Solitons Fractals* 2003;18:141.
- [5] Rosenblum MG, Pikovsky AS, Kurths J. From phase to lag synchronization in coupled chaotic oscillators. *Phys Rev Lett* 1997;78:4193.
- [6] Li CD, Liao XF, Zhang XY. Impulsive synchronization of chaotic systems. *Chaos* 2005;15:023104.
- [7] Moskalenko OI, Koronovskii AA, Hramov AE. Generalized synchronization of chaos for secure communication: remarkable stability to noise. *Phys Lett A* 2010;374:2925.
- [8] Argyris A, Syvridis D, Larger L, et al. Chaos-based communications at high bit rates using commercial fibre-optic links. *Nat* 2006;438:343.
- [9] Koronovskii AA, Moskalenko OI, Hramov AE. On the use of chaotic synchronization for secure communication. *Phys-Usp* 2009;52:1213.
- [10] Li N, Lin XD, Gao ZY, Tang X, Deng T, Ni M, Huang XW, Fan L. Security-enhanced bidirectional communication based on a long-distance chaos synchronization system with double optical feedback. In: 13th international congress on image and signal processing, bioMedical engineering and informatics. vol. 82, 2020.
- [11] Chen LP, Yin H, Huang TW, Yuan LG, Zheng S, Yin LS. Chaos in fractional-order discrete neural networks with application to image encryption. *Neural Netw* 2020;125:174.
- [12] Vaseghi B, Mobayen S, Hashemi SS, Fekih A. Fast reaching finite time synchronization approach for chaotic systems with application in medical image encryption. *IEEE Access* 2021;9:25911.

- [13] Moon S, Baik J-J, Seo JM. Chaos synchronization in generalized Lorenz systems and an application to image encryption. *Commun Nonlinear Sci Numer Simul* 2021;96:105708.
- [14] Jahanshahi H, Yousefpour A, Munoz-Pacheco JM, Kacar S, Pham V-T, Alsaadi FE. A new fractional-order hyperchaotic memristor oscillator: dynamic analysis, robust adaptive synchronization, and its application to voice encryption. *Appl Math Comput* 2020;383:125310.
- [15] Boccaletti S, Kurths J, Osipov G, Valladares DL, Zhou CS. The synchronization of chaotic systems. *Phys Rep* 2002;366:1.
- [16] Lin W, He YB. Complete synchronization of the noise-perturbed Chua's circuits. *Chaos* 2005;15:023705.
- [17] Lin W, Chen GR. Using white noise to enhance synchronization of coupled chaotic systems. *Chaos* 2006;16:013134.
- [18] Xiao YZ, Xu W, Li XC, Tang SF. Adaptive complete synchronization of chaotic dynamical network with unknown and mismatched parameters. *Chaos* 2007;17:033118.
- [19] Pérez-Cruz JH, Portilla-Flores EA, Niño-Suárez PA, Aguilar RV. Design of a nonlinear controller and its intelligent optimization for exponential synchronization of a new chaotic system. *Optik* 2016;130:201–12.
- [20] Rosenblum MG, Pikovsky AS, Kurths J. Phase synchronization of chaotic oscillators. *Phys Rev Lett* 1996;76:1804.
- [21] Guan SG, Lai C-H, Wei GW. Phase synchronization between two essentially different chaotic systems. *Phys Rev E* 2005;72:016205.
- [22] Pikovsky AS, Zaks M, Rosenblum M, Osipov G, Kurths J. Phase synchronization of chaotic oscillations in terms of periodic orbits. *Chaos* 1997;7:680.
- [23] Kocarev L, Parlitz U. Generalized synchronization, predictability, and equivalence of unidirectionally coupled dynamical systems. *Phys Rev Lett* 1996;76:1816.
- [24] Rulkov NF, Sushchik MM, Tsimring LS, Abarbanel HDI. Generalized synchronization of chaos in directionally coupled chaotic systems. *Phys Rev E* 1995;51:980.
- [25] Hramov AE, Koronovskii AA, Popov PV. Generalized synchronization in coupled Ginzburg–Landau equations and mechanisms of its arising. *Phys Rev E* 2005;72:037201.
- [26] Deng WH. Generalized synchronization in fractional order systems. *Phys Rev E* 2007;75:056201.
- [27] Zhang G, Liu Z-R, Ma Z-J. Generalized synchronization of continuous dynamical system. *Appl Math Mech* 2007;28:157.
- [28] Huang YH, Wang Y-W, Xiao J-W. Generalized lag-synchronization of continuous chaotic system. *Chaos Solitons Fractals* 2009;40:766.
- [29] Pyragas K. Weak and strong synchronization of chaos. *Phys Rev E* 1996;54:R4508.
- [30] Barto AG, Sutton RS, Anderson CW. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Trans Syst Man Cybern* 1983;SMC-13(5):834.
- [31] Buşoniu L, Babuška R, Schutter BD. A comprehensive survey of multiagent reinforcement learning. *IEEE Trans Syst Man Cybern C* 2008;38:156.
- [32] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning. *Nat* 2015;518:529.
- [33] Arulkumaran K, Deisenroth MP, Brundage M, Bharath AA. Deep reinforcement learning. *IEEE Signal Process Mag* 2017;34:26.
- [34] Du W, Ding SF. A survey on multi-agent deep reinforcement learning: from the perspective of challenges and applications. *Artif Intell Rev* 2021;54:3215.
- [35] Vinyals O, Babuschkin I, Czarnecki WM, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nat* 2019;575:350.
- [36] Silver D, Schrittwieser J, Simonyan K, et al. Mastering the game of go without human knowledge. *Nat* 2017;550:354.
- [37] Silver D, Hubert T, Schrittwieser J, et al. A general reinforcement learning algorithm that masters chess, shogi and go through self-play. *Sci* 2018;362:1140.
- [38] Jumper J, Evans R, Pritzel A, et al. Highly accurate protein structure prediction with AlphaFold. *Nat* 2021;596:583.
- [39] Kober J, Bagnell JA, Peters J. Reinforcement learning in robotics: a survey. *Int J Robot Res* 2013;32:1238.
- [40] Kiran BR, Sobh I, Talpaert V, Mannion P, Sallab AAA, Yogamani S, Pérez P. Deep reinforcement learning for autonomous driving: a survey. *IEEE Trans Intell Transp Syst* 2022;23:4909.
- [41] Le N, Rathour VS, Yamazaki K, Luu K, Savvides M. Deep reinforcement learning in computer vision: a comprehensive survey. *Artif Intell Rev* 2022;55:2733.
- [42] Martín-Guerrero JD, Lamata L. Reinforcement learning and physics. *Appl Sci* 2021;11:8589.
- [43] Gadaleta S, Dangelmayr G. Optimal chaos control through reinforcement learning. *Chaos* 1999;9:775.
- [44] Bukov M, Day AGR, Sels D, Weinberg P, Polkovnikov A, Mehta P. Reinforcement learning in different phases of quantum control. *Phys Rev X* 2018;8:031086.
- [45] Fösel T, Tighineanu P, Weiss T. Reinforcement learning with neural networks for quantum feedback. *Phys Rev X* 2018;8:031084.
- [46] Cheng HX, Li HH, Dai QL, Yang JZ. A deep reinforcement learning method to control chaos synchronization between two identical chaotic systems. *Chaos Solitons Fractals* 2023;174:113809.
- [47] Zhang J, Yang L, Zhu Q, Grebogi C, Lin W. Machine-learning-coined noise induces energy-saving synchrony. *Phys Rev E* 2024;110:L012203.

- [48] Ding JP, Lei YM. Control of chaos with time-delayed feedback based on deep reinforcement learning. *Phys D* 2023;451:133767.
- [49] Krylov D, Dylov DV, Rosenblum M. Reinforcement learning for suppression of collective activity in oscillatory ensembles. *Chaos* 2020;30:033126.
- [50] Biferale L, Bonaccorso F, Buzzicotti M, Leoni PCD, Gustavsson K. Zermelo's problem: optimal point-to-point navigation in 2D turbulent flows using reinforcement learning. *Chaos* 2019;29:103138.
- [51] Agostinelli F, Hocquet G, Singh S, Baldi P. From reinforcement learning to deep reinforcement learning: an overview. *Lecture Notes in Artificial Intelligence* 2018;11100:298.
- [52] Schmidhuber J. Deep learning in neural networks: an overview. *Neural Netw* 2015;61:85.
- [53] Lecun Y, Bengio Y, Hinton G. Deep learning. *Nat* 2015;521:436.
- [54] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. *arXiv:1707.06347*.
- [55] Fang H-P, Hao B-L. Symbolic dynamics of the Lorenz equations. *Chaos Solitons Fractals* 1996;7:217.
- [56] Yu XH. Variable structure control approach for controlling chaos. *Chaos Solitons Fractals* 1997;8:1577.
- [57] Awrejcewicz J, Calvisi ML. Mechanical models of Chua's circuit. *Int J Bifurc Chaos* 2002;12:671.
- [58] Pérez-Cruz JH, Tamayo-Meza PA, Figueroa M, Silva-Ortigoza R, et al. Exponential synchronization of chaotic xian system using linear feedback control. *Complex* 2019;1:4706491.
- [59] Sun Y-Z, Leng S-Y, Lai Y-C, Grebogi C, Lin W. Closed-loop control of complex networks: A trade-off between time and energy. *Phys Rev Lett* 2017;119:198301.
- [60] Abarbanel HDI, Rulkov NF, Sushchik MM. Generalized synchronization of chaos: the auxiliary system approach. *Phys Rev E* 1996;53:4528.